

Problem Statement Wine Type Classification Using Decision Tree

Problem: Given chemical measurements of wines, build a Decision Tree classifier that predicts the type of wine (3 classes) based on its features.

Objective:

Train a Decision Tree to classify wine types.

Evaluate the model using accuracy, confusion matrix, precision, and recall.

Visualize the Decision Tree for interpretability.

Dataset:

Sklearn's built-in Wine dataset — 178 wine samples, 13 features like alcohol content, phenols, color intensity, etc., and 3 target classes.

Step 0: Import Required Libraries

```
import pandas as pd
import numpy as np

# Dataset
from sklearn.datasets import load_wine

# Train-test split
from sklearn.model_selection import train_test_split

# Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Evaluation metrics
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 1: Load the Wine Dataset

```
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target)

print(X.shape)
print(y.value_counts())
```

```
(178, 13)
1    71
0    59
2    48
Name: count, dtype: int64
```

Step 2: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 3: Train the Decision Tree

```
dt_classifier = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_classifier.fit(X_train, y_train)
```

```
▼      DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier(max_depth=4, random_state=42)
```

Step 4: Make Predictions

```
y_pred = dt_classifier.predict(X_test)
```

Step 5: Evaluate the Model

```
# Accuracy
acc = accuracy_score(y_test, y_pred)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", acc)

# Detailed classification metrics
print(classification_report(y_test, y_pred))
```

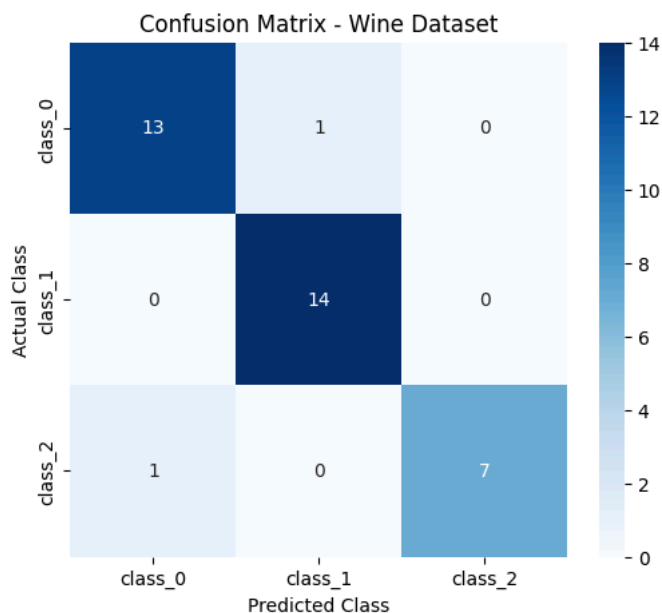
```
Accuracy: 0.9444444444444444
              precision    recall  f1-score   support

     0         0.93      0.93      0.93        14
     1         0.93      1.00      0.97        14
     2         1.00      0.88      0.93         8

   accuracy          0.94          0.94          0.94          36
  macro avg          0.95          0.93          0.94          36
 weighted avg          0.95          0.94          0.94          36
```

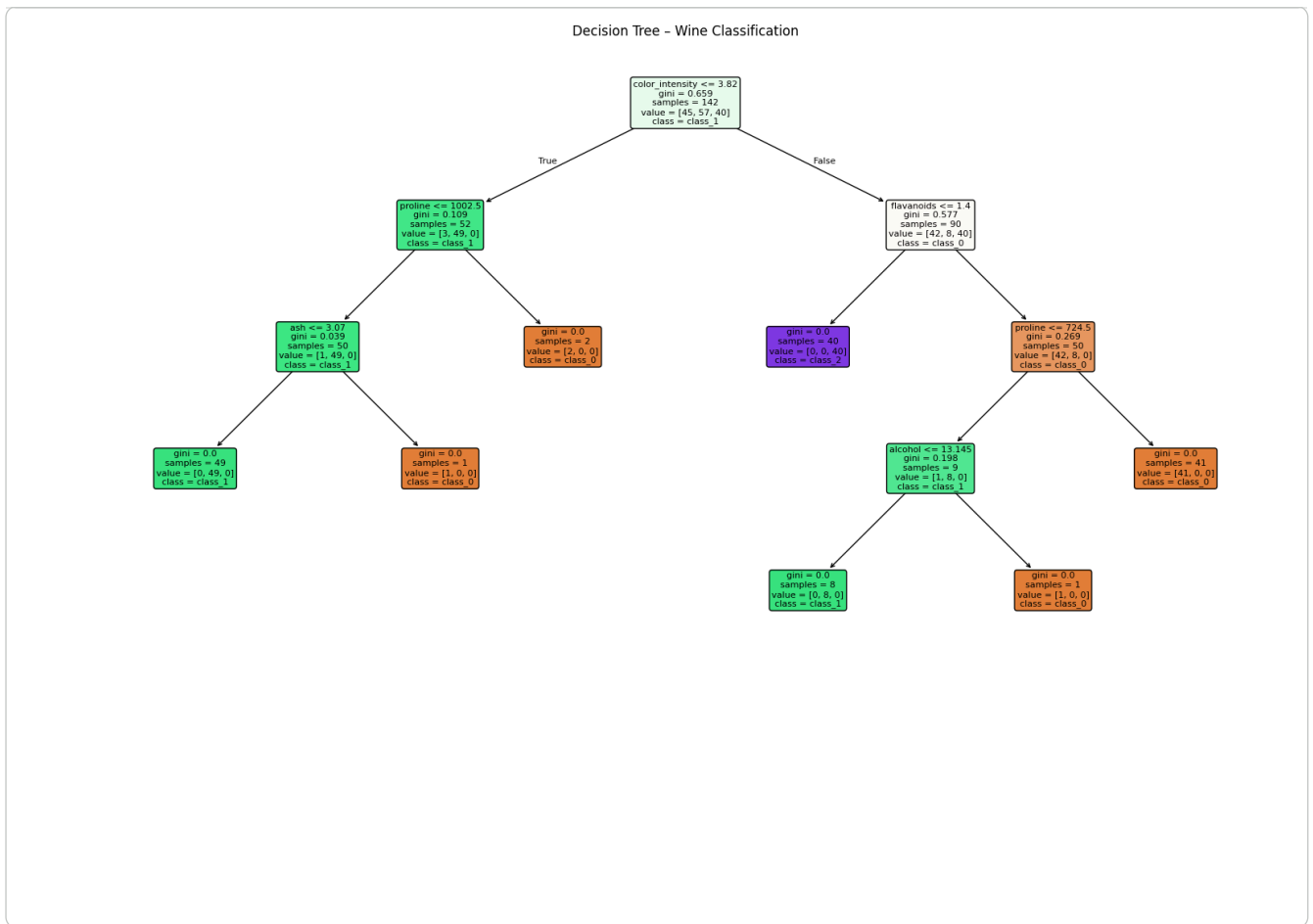
Step 6: Visualize Confusion Matrix

```
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=wine.target_names, yticklabels=wine.target_names)
plt.xlabel('Predicted Class')
plt.ylabel('Actual Class')
plt.title('Confusion Matrix - Wine Dataset')
plt.show()
```



Step 7: Visualize the Decision Tree

```
plt.figure(figsize=(20,10))
plot_tree(dt_classifier, feature_names=wine.feature_names, class_names=wine.target_names,
          filled=True, rounded=True, fontsize=8)
plt.title('Decision Tree - Wine Classification')
plt.show()
```



Decision Tree – Wine Dataset: Steps Summary

Import Libraries

Pandas, Numpy → data handling

Sklearn → dataset, model, train-test split, metrics

Matplotlib & Seaborn → visualizations

Load Dataset

Load the Wine dataset from sklearn.datasets

X = features (13 chemical measurements)

y = target labels (3 wine classes)

Train-Test Split

Split data into training (80%) and testing (20%)

Train Decision Tree Classifier

Use DecisionTreeClassifier(max_depth=4)

Fit model on training data (X_train, y_train)

Make Predictions

Predict target classes for test data (X_test)

Evaluate Model

Metrics: Accuracy, Confusion Matrix, Classification Report

Understand True Positives, False Positives, True Negatives, False Negatives

Visualize Results

Confusion Matrix Heatmap → shows prediction errors visually

Decision Tree Plot → shows tree splits, feature thresholds, and class assignments

Interpret & Insights

Observe most important features used in splits

Understand model behavior and mistakes

Optionally tune tree depth or parameters to avoid overfitting

Double-click (or enter) to edit